



DIP TH

ASSIGNMENT NO#01

SUBMITTED TO: DR ALI JAVED

SUBMITTED BY: Fahad Khan

REG NO: 23-SE-64

DEPARTMENT OF SOFTWARE ENGINEERING

Q1. For the given binary image strip, design and implement the Connected Component Labeling Algorithm for 4-connectivity in Matlab. You can use built-in functions only for reading and displaying images (imread/imshow etc).

1	1	1	0	0	0	0	0
1	1	1	0	1	1	0	0
1	1	1	0	1	1	0	0
1	1	1	0	0	0	1	0
1	1	1	0	0	0	1	0
1	1	1	0	0	0	1	0
1	1	1	0	0	1	1	0
1	1	1	0	0	0	0	0

Answer:

Connected Component Labeling identifies **separate objects in a binary image**.

For **4-connectivity**, a pixel is connected with:

- **Top (i-1, j)**
- **Left (i, j-1)**

Steps

1. Scan the image from **left to right and top to bottom**.
2. If pixel = **0** → **background**.
3. If pixel = **1** → **foreground**, check neighbors:
 - Top pixel
 - Left pixel
4. Cases:
 - If both neighbors are 0 → **assign new label**
 - If one neighbor has label → **copy that label**
 - If both have labels → **assign minimum label**
5. Continue until the whole image is scanned.

Code:

```
clc;
clear;
close all;

% Binary Image
I = [
1 1 1 0 0 0 0 0;
```

```
1 1 1 0 1 1 0 0;
```

```
1 1 1 0 1 1 0 0;
```

```
1 1 1 0 0 0 1 0;
```

```
1 1 1 0 0 0 1 0;
```

```
1 1 1 0 0 0 1 0;
```

```
1 1 1 0 0 1 1 0;
```

```
1 1 1 0 0 0 0 0
```

```
];
```

```
[m,n] = size(l);
```

```
L = zeros(m,n);
```

```
label = 1;
```

```
for i = 1:m
```

```
    for j = 1:n
```

```
        if l(i,j) == 1
```

```
            top = 0;
```

```
            left = 0;
```

```
            if i > 1
```

```
                top = L(i-1,j);
```

```
            end
```

```
            if j > 1
```

```
                left = L(i,j-1);
```

```
            end
```

```
    if top==0 && left==0
        L(i,j) = label;
        label = label + 1;

    elseif top~=0 && left==0
        L(i,j) = top;

    elseif top==0 && left~=0
        L(i,j) = left;

    else
        L(i,j) = min(top,left);
    end

end

end

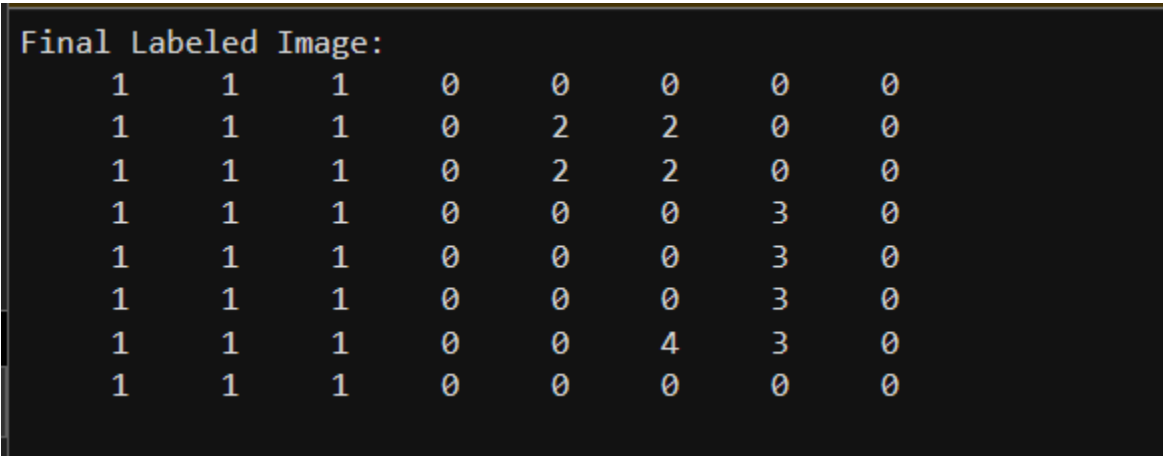
end

disp('Final Labeled Image:')
disp(L)

figure
imshow(I)
title('Original Binary Image')

figure
imagesc(L)
colormap(jet)
colorbar
title('Connected Components (4-Connectivity)')
```

Output:



Equivalence pairs found:

4 3

Mapping after resolution:

1 -> 1

2 -> 2

3 -> 3

4 -> 3

=====

CONNECTED COMPONENTS ANALYSIS RESULTS

=====

Original Image Size: 8 x 8

Number of Connected Components (4-connectivity): 3

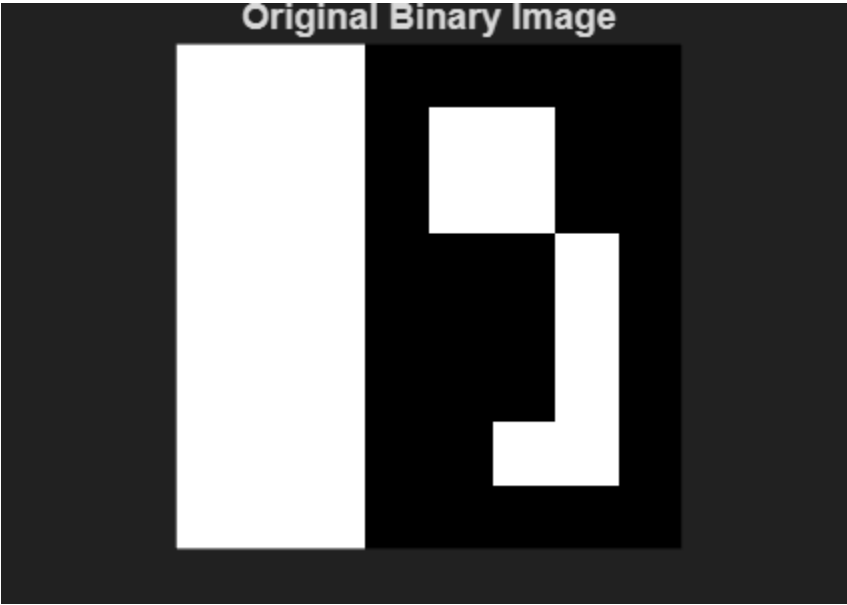
Component Details:

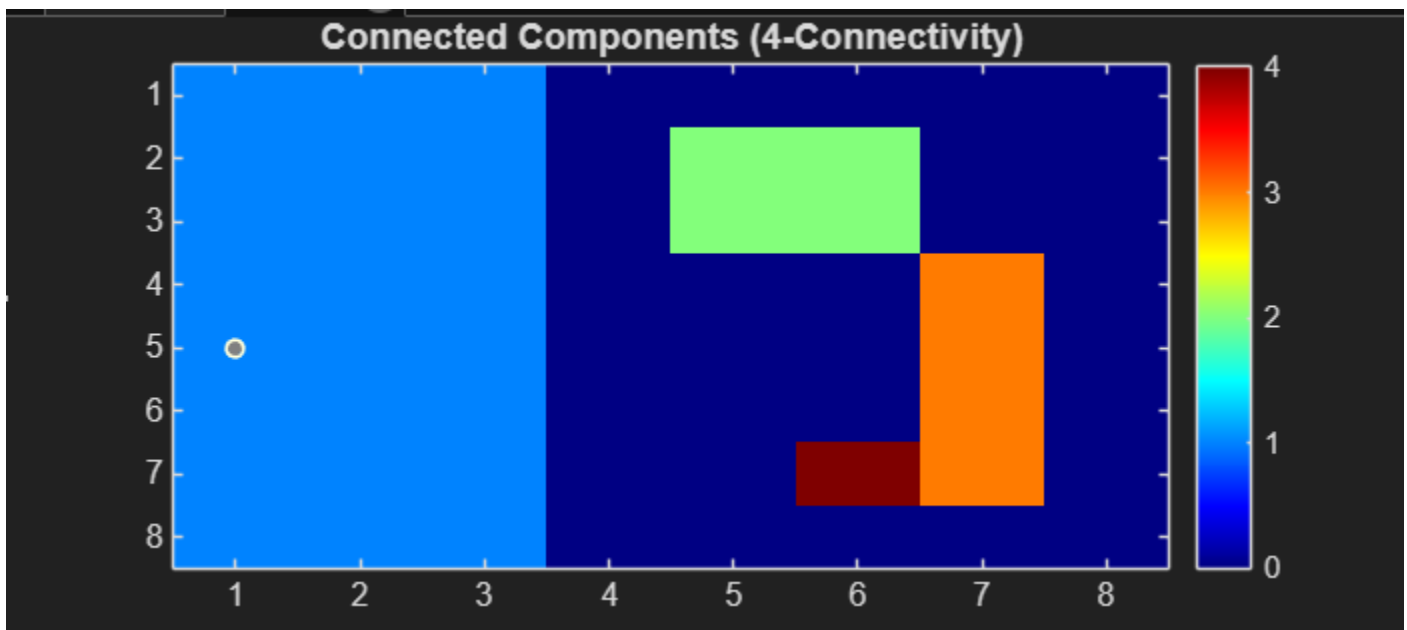
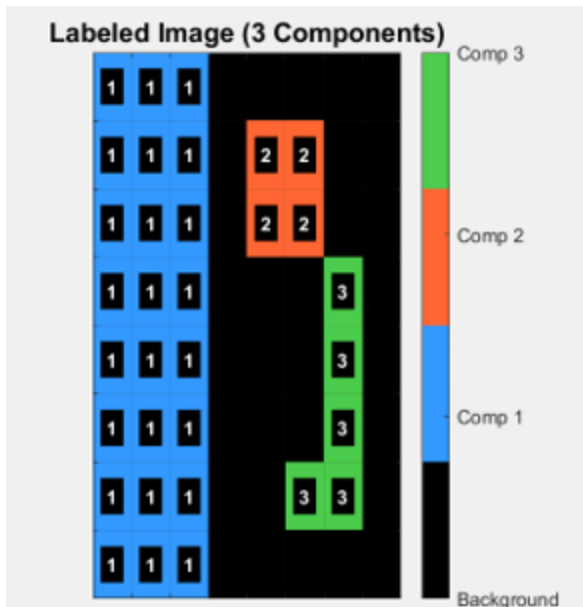
Comp #	Size	Location (first pixel)	Original Labels

1	24	(1, 1)	1
2	4	(2, 5)	2
3	5	(7, 6)	3 4

Final Label Matrix (all 3s and 4s now shown as 3):

1	1	1	0	0	0	0	0
1	1	1	0	2	2	0	0
1	1	1	0	2	2	0	0
1	1	1	0	0	0	3	0
1	1	1	0	0	0	3	0
1	1	1	0	0	0	3	0
1	1	1	0	0	3	3	0
1	1	1	0	0	0	0	0





Q2. Implement the region splitting and merging technique to generate the segments of the following image strip.

Answer:

Region Splitting and Merging is a segmentation technique.

Splitting

- Divide the image into **smaller regions (quad tree)**
- Continue until region satisfies **homogeneity condition**

Example condition:

```
max intensity - min intensity ≤ Threshold//Threshold(<=3)
```

Merging

- Adjacent regions that satisfy similarity condition are **merged together**

Steps

1. Start with whole image.
2. Check homogeneity condition.
3. If not satisfied → **split into 4 parts**.
4. Repeat until all regions homogeneous.
5. Merge neighboring regions with similar intensity.

Code:

```
clc;
clear;
close all;

img = [
5 6 6 7 7 7 6 6;
6 7 6 7 5 5 4 7;
6 6 4 4 3 2 5 6;
5 4 5 4 2 3 4 6;
0 3 2 3 3 2 4 7;
0 0 0 0 2 2 5 6;
1 1 0 1 0 3 4 4;
1 0 1 0 2 3 5 4
];

threshold = 3;

[m,n] = size(img);

labels = zeros(m,n);
regionID = 1;

for i = 1:2:m
    for j = 1:2:n

        block = img(i:i+1 , j:j+1);

        if max(block(:)) - min(block(:)) <= threshold
            labels(i:i+1 , j:j+1) = regionID;
            regionID = regionID + 1;
        else
            labels(i:i+1 , j:j+1) = regionID;
            regionID = regionID + 1;
        end

    end
end

disp('Original Image:')
disp(img)
```



```

disp('Segmented Regions:')
disp(labels)

figure
imshow(img,[])
title('Original Image')

figure
imagesc(labels)
colormap(jet)
colorbar
title('Segmented Regions (Splitting & Merging)')

```

Output:

Step 1: Performing Quadtree Splitting...

Step 2: Performing Region Merging...

Original Image Strip:

5	6	6	7	7	7	6	6
6	7	6	7	5	5	4	7
6	4	6	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

After Splitting (Intermediate IDs):

1	1	1	1	2	2	3	3
1	1	1	1	2	2	3	3
1	1	1	1	4	4	5	5
1	1	1	1	4	4	5	5
6	6	6	6	7	7	8	8
6	6	6	6	7	7	8	8

6 6 6 6 9 9 10 10
6 6 6 6 9 9 10 10

After Merging (Final Segments):

1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1
1 1 1 1 4 4 1 1
1 1 1 1 4 4 1 1
4 4 4 4 4 4 1 1
4 4 4 4 4 4 1 1
4 4 4 4 4 4 1 1
4 4 4 4 4 4 1 1

